

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 状态伪类

- :link伪类表示链接的正常状态, 选择那些尚未被点过的链接
- :visited伪类选择点过的链接
- :hover伪类在用户指针悬停时生效, 不只适用于链接
- :active伪类选择被鼠标指针或触摸操作“激活的”元素, 只发生在鼠标被按下到被释放的这段时间里
- :focus用于选择已经通过指针设备、触摸或键盘获得焦点的元素, 在表单里使用得非常多

```
<!DOCTYPE html>
<html>
<head>
<style type="text/css">
input:focus {background-color:yellow;}
</style>
</head>

<body>
  <form action="form_action.asp" method="get">
    First name: <input type="text" name="fname" /><br />
    Last name: <input type="text" name="lname" /><br />
    <input type="submit" value="Submit" />
  </form>
</body>
</html>
```

输入框获得焦点, 背景色变成黄色

First name:

Last name:

Submit

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-child伪类选择是父元素第一个子元素的指定类型元素
- :last-child伪类选择是父元素最后一个子元素的指定类型元素
- :nth-last-child选择是父元素倒数第n个子元素的指定类型元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-child {font-weight: bold;}
li:first-child {text-transform:uppercase;}
p:last-child {color: red;}
</style>
</head>

<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

①p元素, 是其父元素的第一个子元素

②li元素, 是其父元素的第一个子元素

③p元素, 是其父元素的最后一个子元素

These are the necessary steps:

- INTERT KEY
- Turn key **clockwise**
- Push accelerator

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-child伪类选择是父元素第一个子元素的指定类型元素
- :last-child伪类选择是父元素最后一个子元素的指定类型元素
- :nth-last-child选择是父元素倒数第n个子元素的指定类型元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
li:nth-last-child(2) {color:green;}
</style>
</head>
<body>
  <div>
    <p>These are the necessary steps:</p>
    <ul>
      <li>Intert Key</li>
      <li>Turn key <strong>clockwise</strong></li>
      <li>Push accelerator</li>
    </ul>
    <p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
  </div>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- Turn key **clockwise**
- Push accelerator

Do *not* push the brake at the same time as the accelerator.

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效.</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类（pseudo-class）选择器以冒号(:)为分割添加到选择器末尾，用来选择特定状态下的元素

— 区别

```
<div>
  <h2>这是标题</h2>
  <p>这是段落一</p>
  <p>这是段落二</p>
</div>
```

```
p:first-child
{
  color: red;
}
```

这是标题

这是段落一

这是段落二

```
p:first-of-type {
  color: red;
}
```

这是标题

这是段落一

这是段落二

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :nth-child伪类根据元素在标记中的次序选择相应的元素
- :only-child选择是父元素中唯一子元素的所有元素
- :not伪类选择与参数不匹配的所有元素
- :only-of-type选择在同一个父元素中没有其他相同类型兄弟元素的元素 (“同类型元素数量为1”)

```
<ul>
  <li class="link"><a href="#">United Kingdom</a></li>
  <li class="link"><a href="#">Germany</a></li>
  <li class="link"><a href="#">Finland</a></li>
  <li class="link"><a href="#">Russia</a></li>
  <li class="link2"><a href="#">Spain</a></li>
  <li class="link"><a href="#">Poland</a></li>
</ul>
```



```
li:nth-of-type(2n) {
  background-color: grey;
}
li:nth-of-type(2n+1) {
  background-color: orange;
}
li:not([class="link"]){
  background-color: brown;
}
```



伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- `:nth-child`伪类根据元素在标记中的次序选择相应的元素
- `:only-child`选择是父元素中唯一子元素的所有元素
- `:not`伪类选择与参数不匹配的所有元素
- `:only-of-type`选择在同一个父元素中没有其他相同类型兄弟元素的元素 (“同类型元素数量为1”)

```
<!DOCTYPE html>
<html>
<head>
<style>
p:only-of-type{background:red;}
em:only-child{background:yellow;}
</style>
</head>
<body>
  <div> <p>这是一个段落。</p> </div>
  <div>
    <p>这是一个段落。</p>
    <p>这是一个段落。</p>
  </div>
  <div> <em>这是一个段落。</em> </div>
</body>
</html>
```

这是一个段落。

这是一个段落。

这是一个段落。

这是一个段落。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :nth-child伪类根据元素在标记中的次序选择相应的元素
- :only-child选择是父元素中唯一子元素的所有元素
- :not伪类选择与参数不匹配的所有元素
- :only-of-type选择在同一个父元素中没有其他相同类型兄弟元素的元素 (“同类型元素数量为1”)

```
<div>
  <p>这是段落</p>
  <h2>这是标题</h2>
</div>
```

```
p:only-of-type {
  color: blue;
}
```

这是段落

这是标题

```
p:only-child {
  color: blue;
}
```

这是段落

这是标题

伪类和伪元素选择器

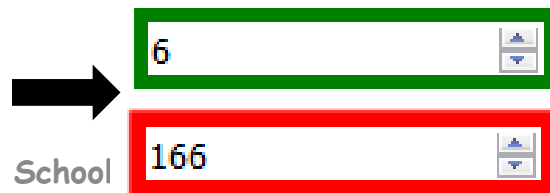
- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 验证伪类

- :checked伪类选择被勾选或选中的单选按钮、多选按钮及列表选项
- :default伪类从表单中选择默认的元素
- :disabled伪类选择禁用状态的表单元素, :enabled
- :empty伪类选择其中不包含任何内容的空元素
- :in-range伪类选择有范围且值在指定范围内的元素, :out-of-range
- :indeterminate伪类选择页面加载时没有勾选的单选按钮或复选框
- :valid伪类选择输入格式符合要求的表单元素, :invalid
- :optional伪类选择表单中非必填的输入字段
- :read-only伪类选择用户不能编辑的元素, :read-write、:required

```
<input type="number" min="5" max="10">
```

```
input[type=number]:in-range { border: 5px solid green; }  
input[type=number]:out-of-range { border: 5px solid red; }
```



伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 语言伪类

- `:lang()` 基于元素语言来匹配页面元素

```
<ol>
  <li lang="en">Manchester United</li>
  <li lang="es">Barcelona</li>
  <li lang="de">Bayern Munich</li>
  <li lang="es">Real Madrid</li>
  <li lang="de">Borussia Dortmund</li>
  <li lang="en">Southampton FC</li>
</ol>
```



```
li:lang("en") {background: pink; font-weight: bold;}
li:lang("de") {background: lightgrey; font-weight: bold;}
li:lang("es") {background: yellow; font-weight: bold;}
```

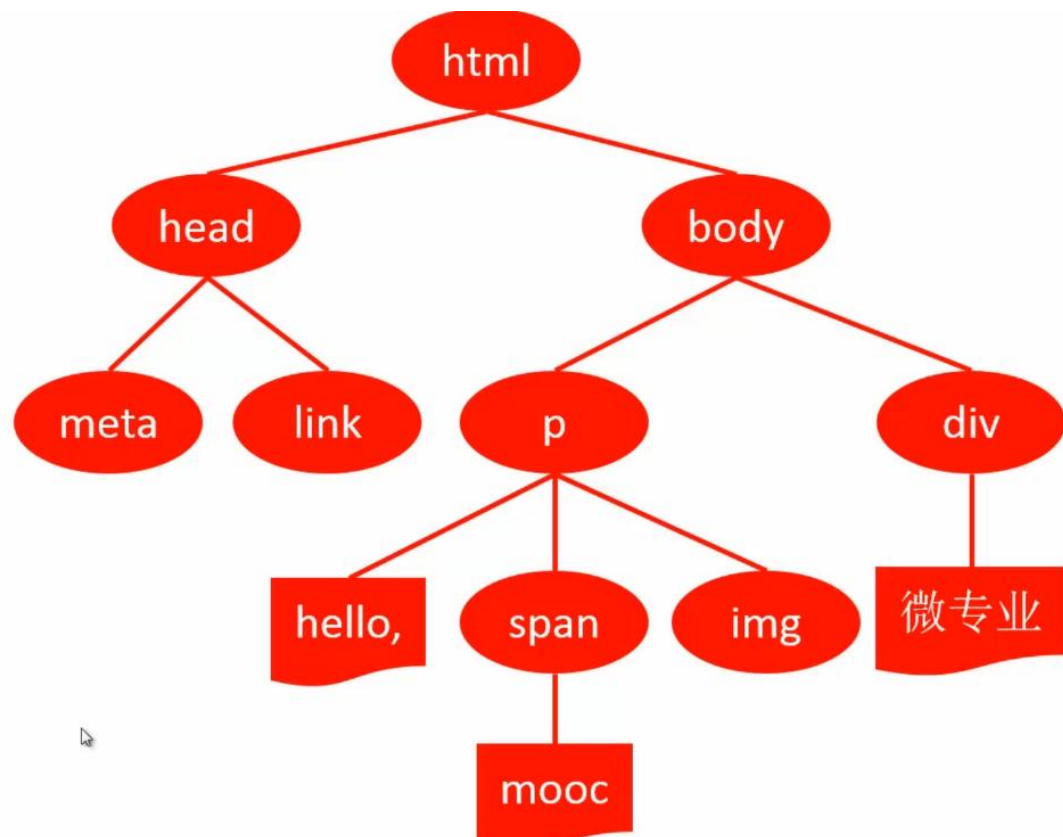
- 
1. Manchester United
 2. Barcelona
 3. Bayern Munich
 4. Real Madrid
 5. Borussia Dortmund
 6. Southampton FC

伪类和伪元素选择器

- DOM树

- DOM（文档对象模型）树是指将HTML或XML文档表示为一个树状结构的方式，其中每个元素、属性、文本和注释都表示为树中的一个节点。

```
<html>
<head>
  <meta charset="utf-8">
  <link href="style.css">
</head>
<body>
  <p>
    hello, <span>mooc</span>
    
  </p>
  <div>微专业</div>
</body>
</html>
```



伪类和伪元素选择器

- 伪元素 (pseudo-element) 选择器以双冒号(::)为分割添加到选择器末尾，用来选择元素的某个特定部分
 - 伪元素可以理解成一种虚拟元素，可以将其视为普通的HTML元素
 - 之所以叫伪元素，是因为伪元素并不存在于DOM树中
 - 伪元素不能在HTML中输入，只能通过CSS创建伪元素

伪类和伪元素选择器

- 伪元素 (pseudo-element) 选择器以双冒号(::)为分割添加到选择器末尾, 用来选择元素的某个特定部分
 - ::before与::after, 为其他HTML元素添加内容 (文本或图形), 添加的内容不实际存在于DOM中, 但可以像存在一样操作它们, 需要在CSS中声明content属性 (表示在某个元素内容的最前面自动插入一段“虚拟的内容”)
 - ::first-letter伪元素选择一行文本第一个字符, 并且文字所处的行之前没有其他内容 (如图片和内联的表格), 只适用于块级元素, 内联元素不适用
 - ::first-line选择元素的第一行, 只适用于块级元素, 内联元素不适用
 - ::selection选择文档中被高亮选中的部分, 基于Gecko的浏览器要使用::-moz-selection

伪类和伪元素选择器

- 伪元素 (pseudo-element) 选择器以双冒号(::)为分割添加到选择器末尾, 用来选择元素的某个特定部分

```
<div>
Great Britain is a large island in the
north Atlantic Ocean off the northwest
coast of continental Europe.
</div>
```

```
div::before{ content:"Note:"; }
div::first-line{font-weight: bold;}
div::first-letter{
    font-size: 1.5em;
    background: red;
}
```

Note: Great Britain is a large island in
the north Atlantic Ocean off the northwest
coast of continental Europe.

提示: ::first-letter 可以选中 ::before 伪元素生成的第一个字符

伪类和伪元素选择器

- 伪元素（pseudo-element）选择器以双冒号(::)为分割添加到选择器末尾，用来选择元素的某个特定部分

No.1:Britain usually refers to the United Kingdom.
No.2:Britain usually refers to the United Kingdom.

```
<!DOCTYPE html>
<html>
<head>
<style type="text/css">
div::before {content:"No.1:"}
span::before {content:"No.2:"}
div::first-letter {color:red}
span::first-letter {color:blue}
</style>
</head>
<body>
<div>Britain usually refers to the United Kingdom.</div>
<span>Britain usually refers to the United Kingdom. </span>
</body>
</html>
```

注意：伪元素的生效范围，**::first-letter**只对块级元素生效，内联元素无效，所以No.2的首字母N不是红色的，而**::before**对块级元素、内联元素没有限制

伪类和伪元素选择器

- 伪元素（pseudo-element）选择器以双冒号(::)为分割添加到选择器末尾，用来选择元素的某个特定部分

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     #p3{color:red;}
7     #p3::selection{
8       color:yellow;
9       background-color:grey;
10    }
11  </style>
12 </head>
13 <body>
14   <p id="p3">文字颜色为红色，选中部分背景色为灰色，文字颜色为黄色</p>
15 </body>
16 </html>
```

显示效果

文字颜色为红色，选中部分背景色为灰色，文字颜色为黄色

通过::selection伪元素，可以调整高亮选文本的样式，本例中调整颜色和背景色

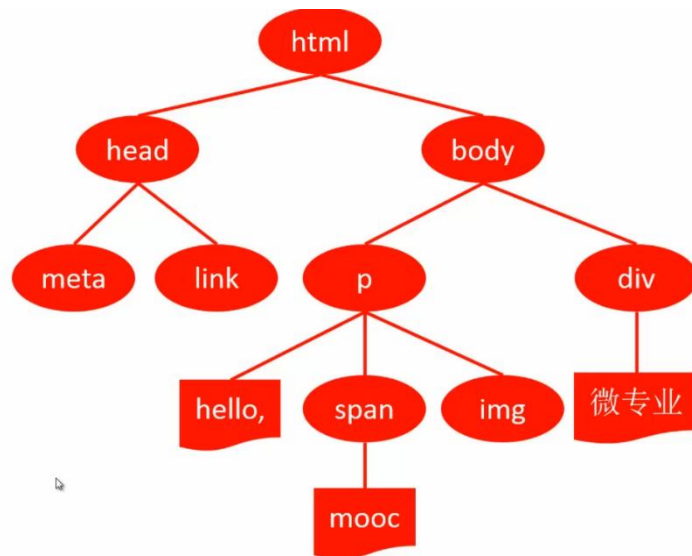
组合选择器和多重选择器

- 基于元素之间的相互关系，CSS提供了几种方法来选择元素，下表使用连接符展示了这些关系(A和B代表前文所述的任意选择器):

名称	组合	Select
群组选择器 Group of selectors	A, B	匹配满足A和B的任意元素
后代选择器 Descendant selector	A B	匹配任意元素，满足条件：B是A的在DOM树上的后代结点（B是A的子节点，或者A的子节点的子节点）
子元素选择器 Child selectors	A > B	匹配任意元素，满足条件：B是A的直接子节点
相邻兄弟选择器 Adjacent sibling selector	A + B	匹配任意元素，满足条件：B是A的下一个兄弟节点（AB有相同的父结点，并且B紧跟在A的后面）
后继兄弟选择器 General sibling selector	A ~ B	匹配任意元素，满足条件：B是A之后的兄弟节点中的任意一个（AB有相同的父节点，B在A之后，但不一定是紧挨着A）

组合选择器和多重选择器

```
<html>
  <head>
    <meta charset="utf-8">
    <link href="style.css">
  </head>
  <body>
    <p>
      hello, <span>mooc</span>
      
    </p>
    <div>微专业</div>
  </body>
</html>
```



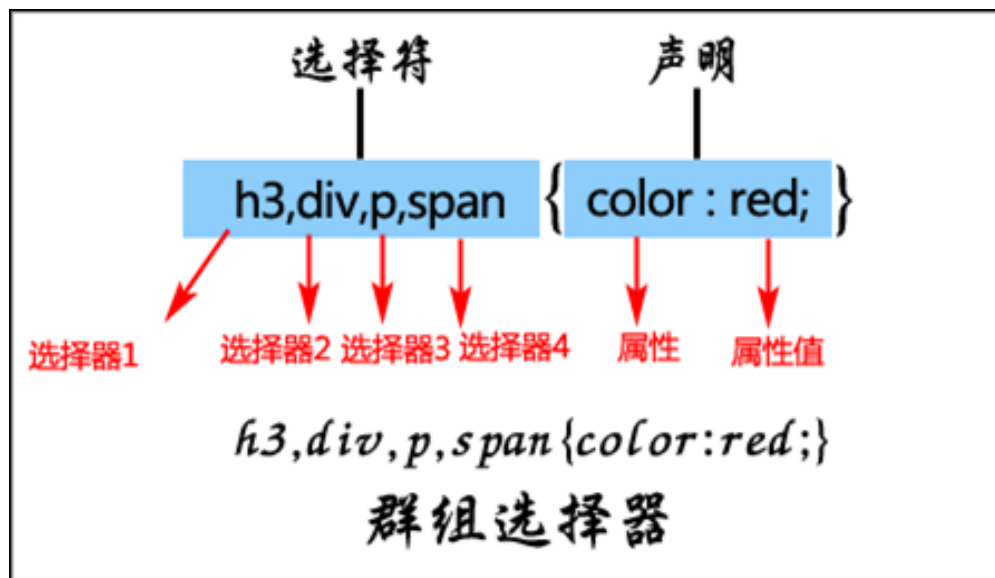
根据元素在文档中的位置确定样式

名称	组合	Select
群组选择器 Group of selectors	A, B	匹配满足A和B的任意元素
后代选择器 Descendant selector	A B	匹配任意元素，满足条件：B是A的在DOM树上的后代结点（B是A的子节点，或者A的子节点的后代结点）
子元素选择器 Child selectors	A > B	匹配任意元素，满足条件：B是A的直接子节点
相邻兄弟选择器 Adjacent sibling selector	A + B	匹配任意元素，满足条件：B是A的下一个兄弟节点（AB有相同的父节点，并且B紧跟在A的后面）
后继兄弟选择器 General sibling selector	A ~ B	匹配任意元素，满足条件：B是A之后的兄弟节点中的任意一个（AB有相同的父节点，B在A之后，但不一定是紧挨着A）

组合选择器和多重选择器

- 群组选择器 (Group of selectors)

- 作用是同时对几个选择器进行相同的操作，解决代码冗长难维护问题
- 对于群组选择器，两个选择器之间必须用“,” (英文逗号) 隔开，不然群组选择器无法生效



组合选择器和多重选择器

- 群组选择器 (Group of selectors)

在线代码测试

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5   <style type="text/css">
6     h3,div,p,span{color:red;}
7   </style>
8 </head>
9 <body>
10  <h3>绿叶学习网</h3>
11  <div>绿叶学习网</div>
12  <p>绿叶学习网</p>
13  <span>绿叶学习网</span>
14 </body>
15 </html>
```

→

显示效果

绿叶学习网
绿叶学习网
绿叶学习网
绿叶学习网

```
1 <style type="text/css">
2   h3,div,p,span{color:red;}
3 </style>
```

=

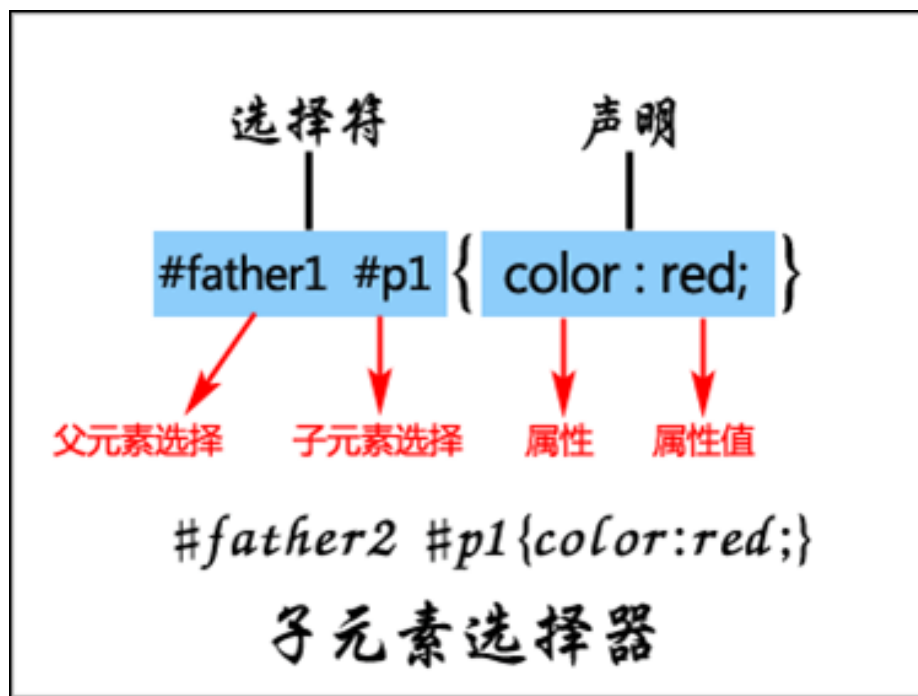
```
1 <style type="text/css">
2   h3{color:red;}
3   div{color:red;}
4   p{color:red;}
5   span{color:red;}
6 </style>
```

• “h3,div,p,span{color:red;}”表示选中所有的h3元素、div元素、p元素和span元素，然后设置这些元素的字体颜色为red

组合选择器和多重选择器

- 后代选择器 (Descendant selector)

- 作用是选中某个元素下的子元素，然后对该子元素设置CSS样式
- 父元素选择器与子元素选择器必须用空格隔开



组合选择器和多重选择器

- 后代选择器（Descendant selector）

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>北邮网安院</title>
6 <style>
7   div p
8   {
9     background-color:yellow;
10  }
11 </style>
12 </head>
13 <body>
14
15 <div>
16 <p>段落 1。 在 div 中。</p>
17 <p>段落 2。 在 div 中。</p>
18 </div>
19
20 <p>段落 3。 不在 div 中。</p>
21 <p>段落 4。 不在 div 中。</p>
22
23 </body>
24 </html>
```

段落 1。 在 div 中。

段落 2。 在 div 中。

段落 3。 不在 div 中。

段落 4。 不在 div 中。

组合选择器和多重选择器

- 后代选择器 (Descendant selector)

在线代码测试

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5   <style type="text/css">
6     #father1 div{color:blue;}
7     #father2 #p1{color:red;}
8   </style>
9 </head>
10 <body>
11   <div id="father1">
12     <div>绿叶学习网</div>
13     <div>绿叶学习网</div>
14   </div>
15   <div id="father2">
16     <p id="p1">绿叶学习网</p>
17     <p id="p2">绿叶学习网</p>
18     <span>绿叶学习网</span>
19   </div>
20 </body>
21 </html>
```

显示效果

绿叶学习网
绿叶学习网
绿叶学习网
绿叶学习网
绿叶学习网

- “#father1 div{...}”表示选择“id为father1的元素”下的所有div元素
- “#father2 #p1{...}”表示选择“id为father2的元素”下的子元素，其中子元素的id为#p1。因为“id为father2的元素”下的第2个p元素没有被选中，所以第2个p元素内容没有变成红色

组合选择器和多重选择器

Descendent selector

Syntax	Example	Example described
<i>selector selector</i>	div strong	 elements that are descendants of a <div>

HTML

```
<div class="hw">
  <h1>Homework 0</h1>
  <p>Due Fri</p>
  <p>Late cutoff Sun</p>
</div>
```

CSS

```
.hw p {
  color: green;
}
```

Homework 0

Due Fri

Late cutoff Sun

Lectures

Apr 3: Syllabus

Apr 5: HTML+CSS

组合选择器和多重选择器

Descendent selector

Syntax	Example	Example described
<i>selector selector</i>	div strong	 elements that are descendants of a <div>

Note: The element does not have to be a direct child. The descendent may be nested many layers in.

The screenshot displays a web browser's developer tools interface. The top section, labeled 'HTML', shows the following structure:

```
<div class="hw">
  <div>
    <p>
      HW0: <strong>Due Friday</strong>
    </p>
  </div>
  HW1 out <strong>Monday</strong>
</div>
```

The bottom section, labeled 'CSS', shows a rule for the class '.hw' applied to 'strong' elements:

```
.hw strong {
  color: red;
}
```

On the right side, the rendered output is shown. The text 'HW0: Due Friday' is in red, and 'HW1 out Monday' is also in red, demonstrating the effect of the CSS rule.

组合选择器和多重选择器

Descendent selector

Syntax	Example	Example described
<i>selector selector</i>	<code>div strong</code>	<code></code> elements that are descendants of a <code><div></code>

Discouraged:

```
<h1 class="hw">Homework 0</h1>
<p class="hw">Due Fri</p>
<p class="hw">Late cutoff Sun</p>
```

vs

Preferred:

```
<div class="hw">
  <h1>Homework 0</h1>
  <p>Due Fri</p>
  <p>Late cutoff Sun</p>
</div>
```

Instead of applying a class to several adjacent elements, wrap the group in a `<div>` container and style the contents via descendent selectors.

组合选择器和多重选择器

- 注意区分后代选择器 (Descendant selector) 和子元素选择器 (Child selectors)
- (A和B代表前文所述的任意选择器):

名称	组合	Select
后代选择器 Descendant selector	A B	匹配任意元素, 满足条件: B是A的在DOM树上的后代结点 (B是A的子节点, 或者A的子节点的后代结点)
子元素选择器 Child selectors	A > B	匹配任意元素, 满足条件: B是A的直接子节点

```
ul > li {  
  border-top: 5px solid red;  
}
```

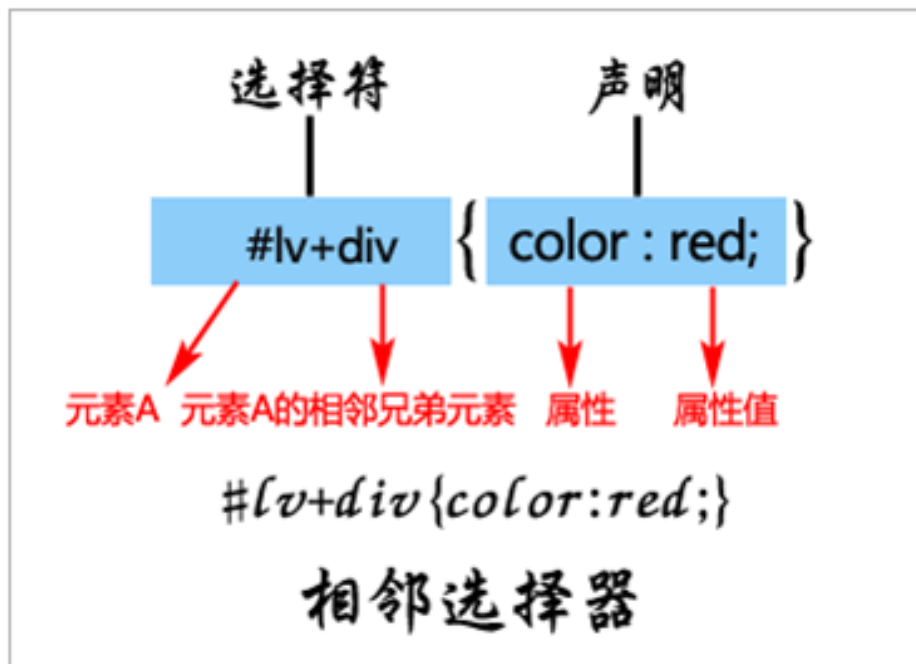
```
<ul>  
  <li>Unordered item</li>  
  <li>Unordered item  
    <ol>  
      <li>Item 1</li>  
      <li>Item 2</li>  
    </ol>  
  </li>  
</ul>
```

- Unordered item
- Unordered item
 - 1. Item 1
 - 2. Item 2

- Unordered item
- Unordered item
 - 1. Item 1
 - 2. Item 2

组合选择器和多重选择器

- 相邻兄弟选择器 (Adjacent sibling selector)
 - 作用是选中某个元素的**相邻兄弟**元素，然后对该元素设置**CSS**样式
 - 元素选择器必须用**加号**隔开



组合选择器和多重选择器

- 相邻兄弟选择器 (Adjacent sibling selector)

在线代码测试	显示效果
<pre>1 <!DOCTYPE html> 2 <html> 3 <head> 4 <title></title> 5 <style type="text/css"> 6 #lv+div{color:red;} 7 </style> 8 </head> 9 <body> 10 <div>绿叶学习网1</div> 11 <div id="lv"> 12 <p>绿叶学习网2</p> 13 </div> 14 <div>绿叶学习网3</div> 15 <div>绿叶学习网4</div> 16 </body> 17 </html></pre>	绿叶学习网1 绿叶学习网2 绿叶学习网3 绿叶学习网4

“#lv+div{...}”表示选择“id为lv的元素”的相邻兄弟元素div，也就是第3个div元素

组合选择器和多重选择器

- 注意区分相邻兄弟选择器 (Adjacent sibling selector) 和后继兄弟选择器 (General sibling selector)
- (A和B代表前文所述的任意选择器):

名称	组合	Select
相邻兄弟选择器 Adjacent sibling selector	A + B	匹配任意元素，满足条件：B是A的下一个兄弟节点 (AB有相同的父结点，并且B紧跟在A的后面)
后继兄弟选择器 General sibling selector	A ~ B	匹配任意元素，满足条件：B是A之后的兄弟节点中的任意一个 (AB有相同的父节点，B在A之后，但不一定是紧挨着A)

```
div~p
{
    background-color:yellow;
}
</style>
```

<p>之前段落，不会添加背景颜色。</p>

<div>

<p>段落 1。 在 div 中。</p>

<p>段落 2。 在 div 中。</p>

</div>

<p>段落 3。 不在 div 中。</p>

<p>段落 4。 不在 div 中。</p>

之前段落，不会添加背景颜色。

段落 1。 在 div 中。

段落 2。 在 div 中。

段落 3。 不在 div 中。

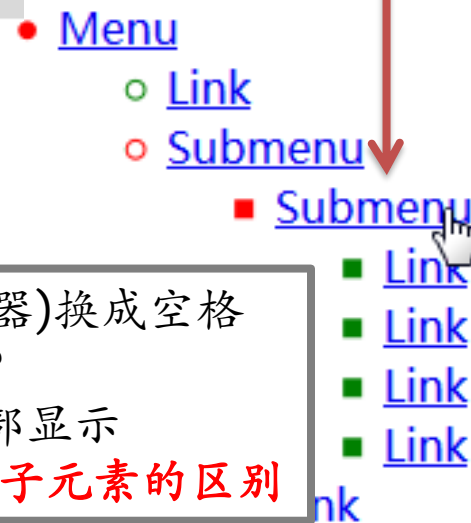
段落 4。 不在 div 中。

组合器和多重选择器

• 示例：利用组合器和伪类实现下拉菜单

```
<div class="menu-bar">
<ul>
  <li>
    <a href="example.html">Menu</a>
    <ul>
      <li>
        <a href="example.html">Link</a>
      </li>
      <li>
        <a class="menu-nav" href="example.html">Submenu</a>
        <ul>
          <li>
            <a class="menu-nav" href="example.html">Submenu</a>
            <ul>
              <li><a href="example.html">Link</a></li>
              <li><a href="example.html">Link</a></li>
              <li><a href="example.html">Link</a></li>
              <li><a href="example.html">Link</a></li>
            </ul>
          </li>
          <li><a href="example.html">Link</a></li>
        </ul>
      </li>
    </ul>
  </li>
</ul>
</div>
```

```
div.menu-bar ul ul {
  display: none;
}
div.menu-bar li:hover {
  color: red;
}
div.menu-bar li:hover > ul {
  display: block;
  color: green;
}
```



尝试：把>(子元素选择器)换成空格
(后代选择器)会怎么样？

A：下拉菜单会一次全部显示

这个例子表现了后代和子元素的区别

通配选择器

- 星号(*)是通配选择器，用来匹配任意类型的HTML元素

```
<div>
  <p>I think the containing box just needed
  a <strong>border</strong> or <em>something</em>,
  but this is getting <strong>out of
hand</strong>!</p>
</div>
```

```
*{ color:red; }
*:not(strong){ color:green;}
* em{ color:purple; }
```

I think the containing box just needed a **border** or *something*, but this is getting **out of hand**!

提示:在配合其他简单选择器的时候，省略掉通配选择器会有同样的效果，比如，*em和em的效果完全相同。由于适用于所有的元素，在大型网页中会**对性能有明显影响，请谨慎使用**

身边的CSS实例

- 北邮网络空间安全学院

– <https://scss.bupt.edu.cn/index.htm>

```
{ } 样式编辑器 性能 内存 网络 存储 无障碍环境 新 ABP Adblock Plus <> DOM
1 > /** 通用样式 */
2 > body, h1, h2, h3, h4, h5, h6, hr, p, blockquote, dl, dt, dd, ul, ol, li, pre, form, fieldset, legend,
3 > body, button, input, select, textarea { font: 12px/1.5 'Simsun', tahoma, Srial, helvetica, sans-serif; }
4 > h1, h2, h3, h4, h5, h6 { font-size: 100%; font-weight: normal; }
5 > em { font-style: normal; }
6 > ul, ol { list-style: none; }
7 > a { text-decoration: none; color: #333; outline: none; }
8 > a: hover { text-decoration: none; }
9 > fieldset, img { border: medium none; vertical-align: middle; }
10 > body a { outline: none; blr: expression(this.onFocus=this.blur()); }
11 > a: focus { outline: none; -moz-outline: none; }
12 > a: hover { text-decoration: none; color: #0b6cb8; -moz-transition: all 0.3s ease-in; -webkit-transition: a
13 > /*li: hover { -moz-transition: all 0.3s ease-in; -webkit-transition: all 0.3s ease-in; -o-transition: al
14 > /*火狐去掉按钮、连接的虚线*/
15 > a: focus, a: active,
16 > button: : -moz-focus-inner,
17 > input [type="reset"] : : -moz-focus-inner,
18 > input [type="button"] : : -moz-focus-inner,
19 > input [type="submit"] : : -moz-focus-inner,
20 > select : : -moz-focus-inner,
21 > input [type="file"] > input [type="button"] : : -moz-focus-inner {
22 > border: 0;
23 > outline : 0;
24 > }
```

选择器小结

- 常见的CSS选择器

Example	Description
<code>p</code>	All <code><p></code> elements
<code>.abc</code>	All elements with the abc class , i.e. <code>class="abc"</code>
<code>#abc</code>	Element with the abc id , i.e. <code>id="abc"</code>
<code>p.abc</code>	<code><p></code> elements with abc class
<code>p#abc</code>	<code><p></code> element with abc id (<code>p</code> is redundant)
<code>div strong</code>	<code></code> elements that are descendants of a <code><div></code>
<code>h2, div</code>	<code><h2></code> elements and <code><div></code> s

选择器小结

- 如何解读复杂的CSS选择器

```
#main li.important strong {  
    color: red;  
}
```

A: Read from right to left:

- `<strong>` tags that are children of `<li>` tags that have an "important" class that are children of the element with the "main" id.

选择器小结

- 如何解读复杂的CSS选择器

```
/* selects any <span> that is inside a <p>, which is inside an <article> */  
article p span { ... }
```

```
/* selects any <p> that comes directly after a <ul>, which comes directly after an <h1> */  
h1 + ul + p { ... }
```

```
body h1 + p .special {  
    color: yellow;  
    background-color: black;  
    padding: 5px;  
}
```

在<body>之内，紧接在<h1>后面的<p>元素的内部，类名为 special 的元素

选择器小结

- 避免常见错误

p.abc **vs** p .abc

p .abc **vs** p, .abc

选择器小结

- 避免常见错误

p.abc **vs** p .abc

- A <p> element with the **abc** class **vs**
An element with the **abc** class that descends from <p>

p .abc **vs** p, .abc

- An element with the **abc** class that descends from <p> **vs**
All <p> elements *and* all elements with the **abc** class

接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

CSS样式概览

- 总计超过500种CSS的样式属性
 - <https://developer.mozilla.org/zh-CN/docs/Web/CSS/Reference>

文本和段落样式

- 用于设置文本和段落样式的 CSS 属性通常可分为两类：

— 文本的展示效果

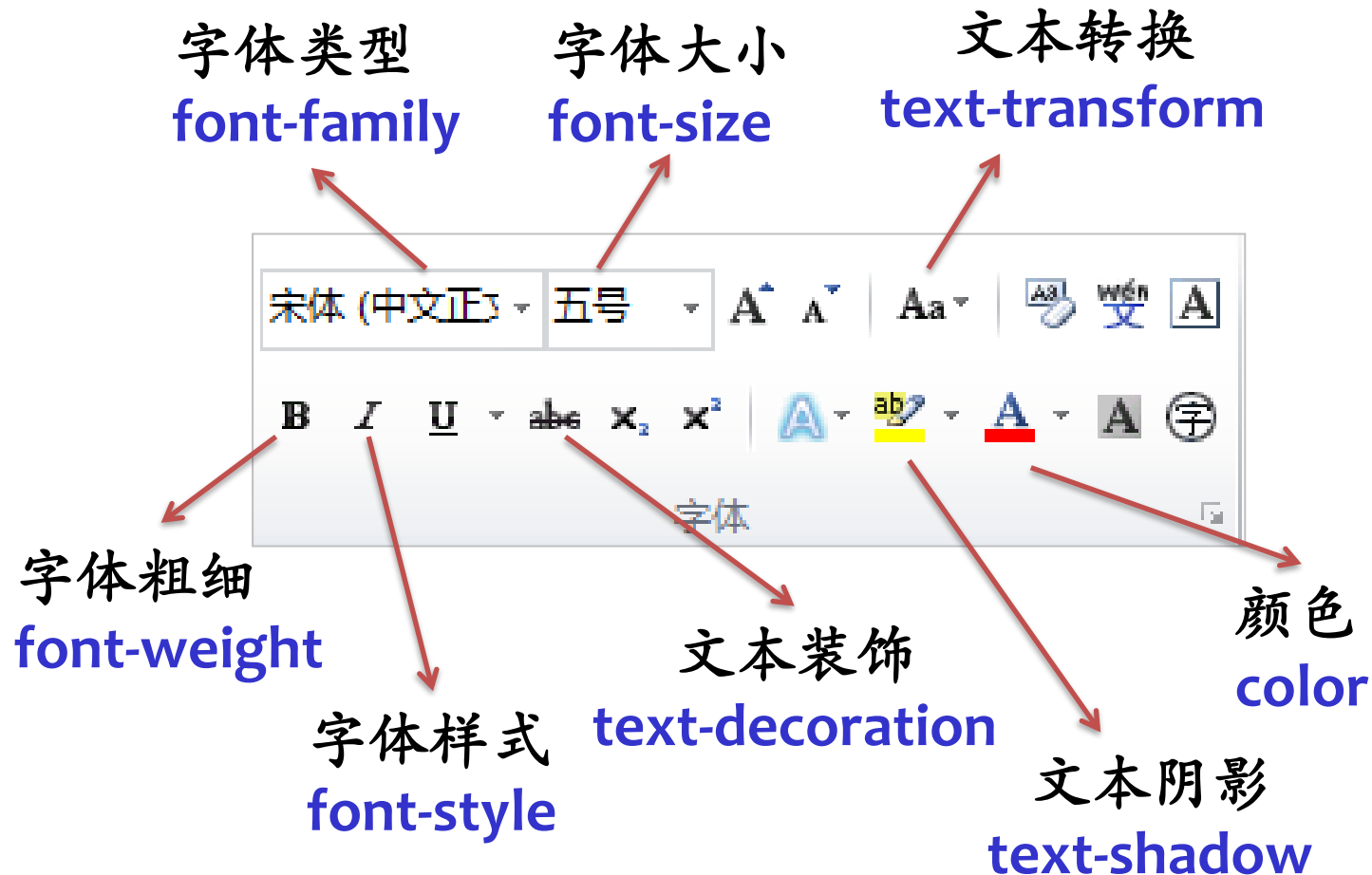
- 用于设置字体、字号、阴影等样式，会直接应用到文本中，比如使用哪种字体、字体的大小是怎样的、字体是粗体还是斜体等

— 文本的布局风格

- 用于设置文本的间距以及其他布局功能，比如设置行与字之间的空间，以及在内容框中文本如何对齐等

文本和段落样式

- 常见的文本展示效果相关属性：



文本和段落样式

- 字体类型font-family

- 在CSS中，使用font-family属性来定义字体类型
- font-family可指定多种字体，多个字体将按优先顺序排列，以逗号隔开，注意逗号一定要是英文逗号

- 语法

- font-family: 字体1,字体2,字体3;

在线代码测试	黑暗 黎明	显示效果
<pre>1 <!DOCTYPE html> 2 <html> 3 <head> 4 <title></title> 5 <style type="text/css"> 6 #p1{font-family:宋体;} 7 #p2{font-family:微软雅黑;} 8 </style> 9 </head> 10 <body> 11 <p id="p1">字体为宋体</p> 12 <p id="p2">字体为微软雅黑</p> 13 </body> 14 </html></pre>		字体为宋体 字体为微软雅黑

文本和段落样式

• 字体类型font-family

- 通过给定一个有先后顺序的，由特定字体或通用字体组成的列表(font stack)来为选定的元素设置字体
- 如果字体不可用，则用浏览器默认的字体代替
- 当使用通用字体时，使用的字体取决于每个浏览器

```
/* 一个字体族名和一个通用的字体族名 */  
font-family: Gill Sans Extrabold, sans-serif;  
font-family: "Goudy Bookletter 1911", sans-serif;
```

```
/* 仅有通用的字体族名 */
```

```
font-family: serif;  
font-family: sans-serif;  
font-family: monospace;  
font-family: cursive;  
font-family: fantasy;
```

```
/* 全局值 */
```

```
font-family: inherit;  
font-family: initial;  
font-family: unset
```

有关通用字体:

http://www.w3school.com.cn/css/css_font-family.asp

<http://www.runoob.com/css/css-font.html>

Generic family	字体系列	说明
Serif	Times New Roman Georgia	Serif字体中字符在行的末端拥有额外的装饰
Sans-serif	Arial Verdana	"Sans"是指无 - 这些字体在末端没有额外的装饰
Monospace	Courier New Lucida Console	所有的等宽字符具有相同的宽度

F

Sans-serif

F

Serif

F

Serif
(red serifs)

irity, BUPT #49

文本和段落样式

- 字体大小font-size

- 语法:

- font-size: 关键字/值;

- 绝对大小或相对大小关键字列表中的某个关键字

- 绝对大小关键字: xx-small, x-small, small, medium, large, x-large, xx-large
 - 相对大小关键字: larger, smaller, 相对父元素

- 使用像素px或em数字值或百分比等

- 像素px, 浏览器默认字体大小是16 像素
 - em值, 1em等于1倍继承字体大小
 - 百分比, 相对于继承字体大小的百分比

文本和段落样式

- 字体大小font-size

- 使用像素px或em数字值或百分比等

- 像素(px):像素是固定大小的单元,用于屏幕媒体(即在电脑屏幕上读取)。一个像素等于电脑屏幕上的一个点(是你屏幕分辨率的最小分割)。许多网页设计师在web文档使用像素单位以生产浏览器渲染的像素完美呈现的网站。像素单元的一个问题是,它没有为视障读者的扩展,以适应移动设备
 - 点(pt):点通常用于印刷媒体(任何打印在纸上的媒体,等等)。一个点等于一英寸的1/72。点更像像素,他们是固定大小的单位,不能伸缩

文本和段落样式

- 字体大小font-size

- 使用像素px或em数字值或百分比等

- “Ems”(em): “em”是一个可伸缩的单位, 用于web文档媒体展示。一个em等于当前的字体大小,例如,如果文档的字体大小是12 pt,1 em等于12 pt。Ems在本质上是可伸缩的,所以2 em相当于24 pt, 0.5 em相当于6 pt等。ems由于其可伸缩性和适应移动设备的特性在web文档中正变得越来越受欢迎
 - 百分比(%):百分比单位更像“em”单位, 当前的字体大小等于100%(比如12 pt = 100%)。当使用百分比单位,你的文字在移动设备上仍然保持完全的可伸缩性和可访问性。

文本和段落样式

- 字体大小font-size

单位	类型	依赖关系	常见应用场景
em	相对单位	依赖父元素的字体大小	响应式设计、相对尺寸控制
px	绝对单位	固定像素，不依赖其他元素	精确控制尺寸、屏幕设计
pt	绝对单位	固定尺寸，1pt = 1/72英寸	打印设计

在Web开发中，**px**用于精确控制，而**em**则常用于创建灵活的布局和响应式设计。**pt**主要用于打印场景，因此在网页设计中使用较少。

文本和段落样式

- 字体大小font-size

```
<p class="xx-small">xx-small P</p>
<p class="medium">medium P</p>
<p class="x-large">x-large P</p>
<p class="default">Default P</p>
<p class="point">16 point P</p>
<p class="larger">larger P</p>
<p class="percent">200% P</p>
<p class="em">2em P</p>
```



```
.xx-small {font-size: xx-small;}
.x-large {font-size: x-large;}
.medium {font-size: medium;}
.larger {font-size: larger;}
.point {font-size: 16px;}
.percent {font-size: 200%;}
.em {font-size: 2em;}
```

xx-small P

medium P

x-large P

Default P

16 point P

larger P

200% P

2em P

缺省大小是16px

都是继承大小
的两倍

文本和段落样式

- 字体粗细**font-weight**

- 语法:

- **font-weight:粗细值;**

- 取值:

- 关键字, **normal**、**lighter**、**bold**、**bolder**
 - 100, 200, ..., 900数值, 值越大就表示越粗, 值越小就表示越细。通常使用**normal (=400)**、**bold(=700)**

文本和段落样式

- 字体粗细font-weight
 - 中文网页常用bold、normal这两个属性值

在线代码测试

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5   <style type="text/css">
6     #p1{font-weight:lighter;}
7     #p2{font-weight:normal;}
8     #p3{font-weight:bold;}
9     #p4{font-weight:bolder;}
10  </style>
11 </head>
12 <body>
13   <p id="p1">字体粗细为:lighter</p>
14   <p id="p2">字体粗细为:normal(正常字体) </p>
15   <p id="p3">字体粗细为:bold</p>
16   <p id="p4">字体粗细为:bolder </p>
17 </body>
18 </html>
```

显示效果

字体粗细为:lighter

字体粗细为:normal(正常字体)

字体粗细为:bold

字体粗细为:bolder

文本和段落样式

- 字体样式font-style

- 语法

- font-style: 属性值;

表1 font-style属性

font-style属性值	说明
normal	默认值，正常体
italic	斜体，这是一个属性
oblique	将字体倾斜，将没有斜体变量 (italic) 的特殊字体，要应用oblique

- italic和oblique 区别

- 有些字体有斜体属性，有些字体没有斜体属性。italic是使用文字的斜体，oblique是让没有斜体属性的文字倾斜

文本和段落样式

- 字体样式font-style

在线代码测试

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title></title>
5     <style type="text/css">
6         #p1{font-style:normal;}
7         #p2{font-style:italic;}
8         #p3{font-style:oblique;}
9     </style>
10 </head>
11 <body>
12     <p id="p1">字体样式为normal</p>
13     <p id="p2">字体样式为italic </p>
14     <p id="p3">字体样式为oblique</p>
15 </body>
16 </html>
```

显示效果

字体样式为normal

字体样式为*italic*

字体样式为*oblique*

文本和段落样式

- 颜色color

- 语法:

- color:颜色值;

- 设置选中元素的**前景内容**的颜色(通常指文本, 也包含文本下方或上方的线)

在线代码测试	显示效果
<pre>1 <!DOCTYPE html> 2 <html> 3 <head> 4 <title></title> 5 <style type="text/css"> 6 #p1{color:gray;} 7 #p2{color:orange;} 8 #p3{color:red;} 9 </style> 10 </head> 11 <body> 12 <p id="p1">字体颜色为灰色</p> 13 <p id="p2">字体颜色为橙色</p> 14 <p id="p3">字体颜色为红色</p> 15 </body> 16 </html></pre>	字体颜色为灰色 字体颜色为橙色 字体颜色为红色

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称

颜色类型	示例
十六进制颜色 所有主要浏览器都支持十六进制颜色值。指定一个十六进制的颜色其组成部分是：#RRGGBB，其中RR（红色），GG（绿色）和BB（蓝色）。所有值必须介于0和FF之间	<pre>p{background-color: #0000ff;}</pre>  #0000ff值呈现为蓝色，因为蓝色的组成设置为最高值（FF）而其他设置为0

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称

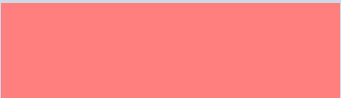
颜色类型	示例
RGB颜色 所有主要浏览器都支持。 RGB颜色值指定： RGB（红，绿，蓝） 每个参数（红色，绿色和蓝色）定义颜色的亮度，可在0和255之间，或一个百分比值（从0%到100%）之间的整数	<pre>p{ background-color: rgb(0,0, 255); }</pre>  RGB（0,0,255）值呈现为蓝色，因为蓝色的参数设置为最高值（255）而其他设置为0 RGB(0,0,255)=RGB(0%，0%，100%)

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称

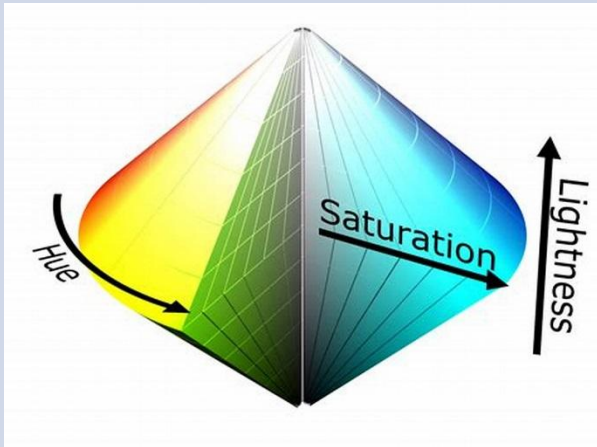
颜色类型	示例
RGBA颜色 RGBA颜色值被IE9, Firefox3+, Chrome, Safari, 和Opera10+ 支持 RGBA颜色值是RGB颜色值alpha通道的延伸 - 指定对象的透明度 RGBA颜色值指定: RGBA (红, 绿, 蓝, alpha)。Alpha参数是一个介于0.0 (完全透明) 和1.0 (完全不透明) 之间的参数	<pre>p{background-color:rgba(255,0,0,0.5);}</pre> 半透明红色: 

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称

颜色类型	示例
HSL颜色 IE9, Firefox, Chrome, Safari, 和Opera 10+. 支持HSL颜色值。 HSL颜色值指定: HSL (色相, 饱和度, 明度) 色相是在色轮上的程度 (从0到360) , 0 (或360) 是红色的, 120是绿色的, 240是蓝色的。 饱和度是一个百分数, 0%意味着灰色, 100%是全彩 明度也是一个百分数, 0%是黑色的, 100%是白色的	<pre>p{ background-color:hsl(120,65%,75%); }</pre> 

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称

颜色类型	示例
HSLA颜色 HSLA颜色值被IE9, Firefox3+, Chrome, Safari, 和Opera10+.支持. HSLA的颜色值是一个带有alpha通道的HSL颜色值的延伸, 指定对象的透明度。 指定HSLA颜色值: HSLA (色调, 饱和度, 亮度, α) α是Alpha参数定义的不透明度。Alpha参数是一个介于0.0 (完全透明) 和1.0 (完全不透明) 之间的参数。	<pre>p{ background-color: hsla(120,65%,75%,0.3); }</pre> 

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)

- 十六进制颜色、RGB颜色、RGBA颜色、HSL色彩
 - HSLA颜色、预定义/跨浏览器的颜色名称（140种）

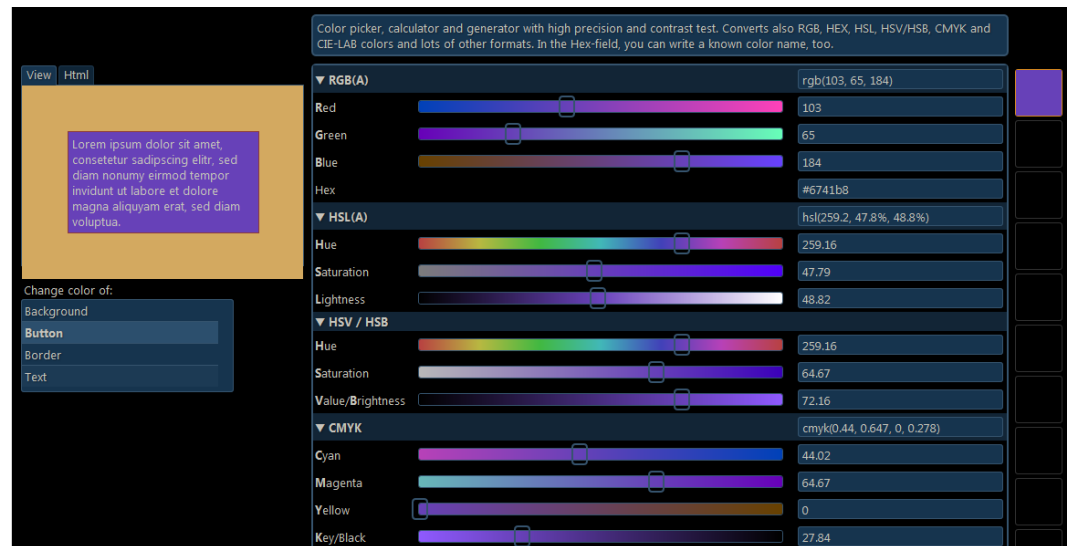
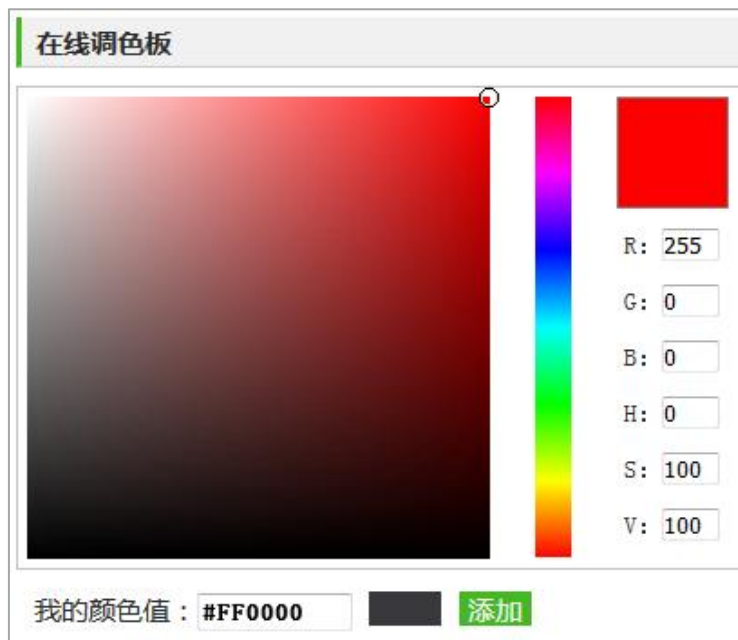
<http://www.runoob.com/cssref/css-colornames.html>

Color Name	HEX	Color
AliceBlue	#F0F8FF	
AntiqueWhite	#FAEBD7	
Aqua	#00FFFF	
Aquamarine	#7FFFD4	
Azure	#F0FFFF	
Beige	#F5F5DC	
Bisque	#FFE4C4	
Black	#000000	
BlanchedAlmond	#FFEBCD	
Blue	#0000FF	
BlueViolet	#8A2BE2	
Brown	#A52A2A	
BurlyWood	#DEB887	

文本和段落样式

- 颜色color

- CSS中颜色取值参见[CSS 合法颜色值](#)
- 实际使用: 在线调色板, Color picker



http://www.lvystudy.com/tools/color_picker.aspx

<http://colorizer.org/>

美国对我开展的持续性网络渗透行动

- 10月19日上午，国家安全机关披露了美国国家安全局（以下简称NSA）对国家授时中心（以下简称“授时中心”）实施重大网络攻击活动。
 - 2022年3月起，NSA利用某国外品牌手机短信服务漏洞，秘密监控10余名国家授时中心工作人员，非法窃取手机通讯录、短信、相册、位置信息等数据。
 - 2023年4月起，NSA在“三角测量”行动曝光前，多次于北京时间凌晨，利用在某国外品牌手机中窃取的登录凭证入侵国家授时中心计算机，刺探内部网络建设情况。
 - 2023年8月至2024年6月，NSA针对性部署新型网络作战平台，对国家授时中心多个内部业务系统实施渗透活动，并企图向高精度地基授时导航系统等重大科技基础设施发动攻击。
 - NSA使用的网攻武器共计42款，可分为三类：前哨控守、隧道搭建和数据窃取
- NSA在战术理念、操作手法、加密通讯、免杀逃逸等方面依然表现出世界领先水准。**隐匿实施攻击**，NSA通过使用正常业务数字证书、伪装Windows系统模块、代理网络通信等方式隐蔽其攻击窃密行为，同时对杀毒软件机制的深入研究，可使其有效避免检测；**通讯多层加密**，NSA使用网攻武器构建回环嵌套加密模式，加密强度远超常规TLS通讯，通信流量更加难以解密还原；**活动耐心谨慎**，在整个活动周期，NSA会对受控主机进行全面监控，文件变动、关机重启都会导致其全面排查异常原因；**功能动态扩展**，NSA会根据目标环境，动态组合不同网攻武器功能模块进行下发，表明其统一攻击平台具备灵活的可扩展性和目标适配能力。

灵魂拷问

❖ 我们发现美国对我国的攻击证据，主要靠



❖ 美国发现我国攻击他们的证据，主要靠

污蔑

中国坚决反对并依法打击任何形式的黑客攻击，更不会对黑客攻击进行鼓励支持或纵容。